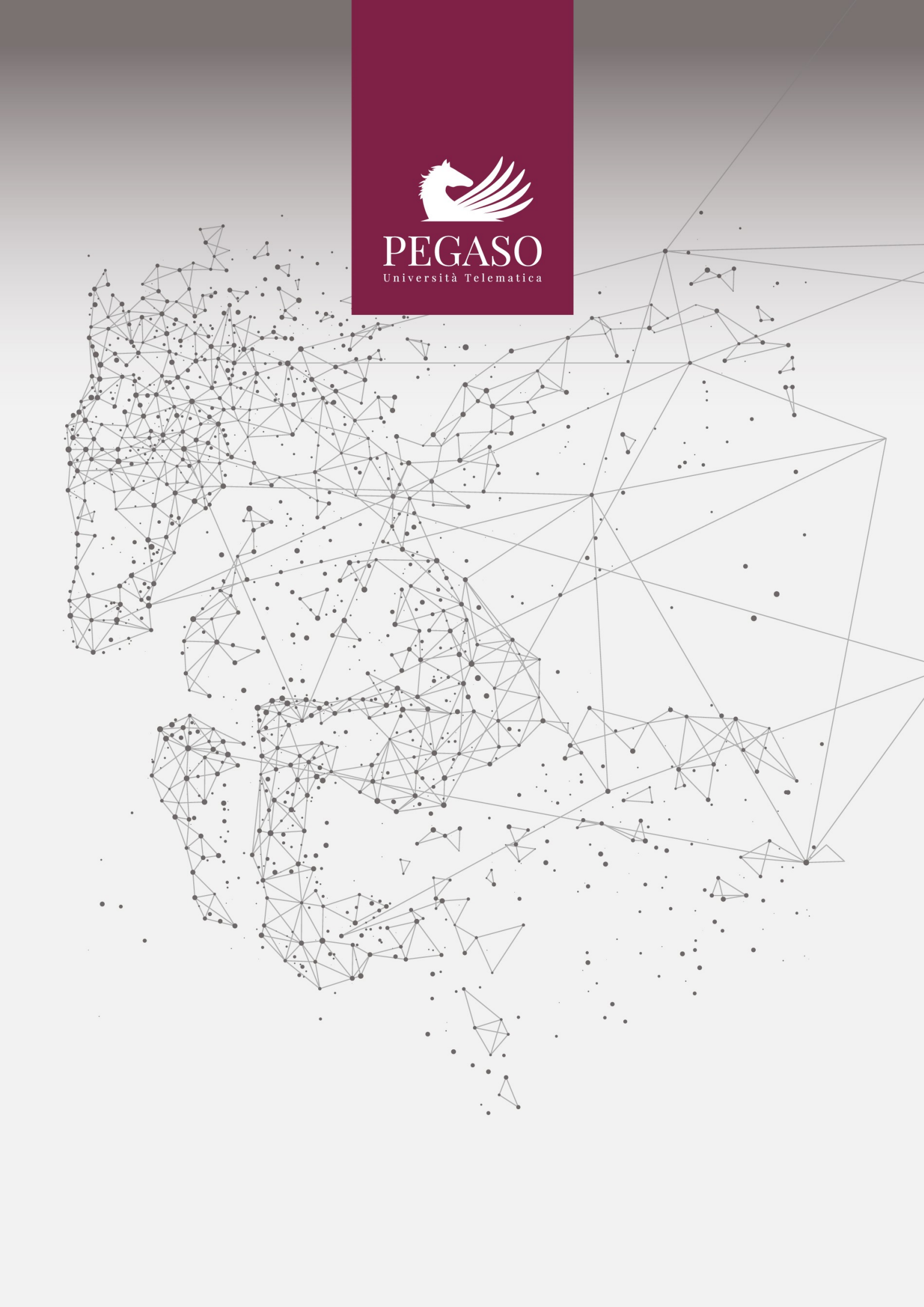

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. UN PO' DI STORIA…. 3
2. PROPRIETÀ DEL LINGUAGGIO JAVA 5
3. JAVA PROGRAMMING MODEL 7
4. IL PRIMO PROGRAMMA IN JAVA 9
5. RACCOMANDAZIONI PER CHI VIENE DALLA PROGRAMMAZIONE PROCEDURALE 12
6. SCRIVERE, COMPILARE ED ESEGUIRE UN PROGRAMMA IN JAVA 14
7. AMBIENTE DI SVILUPPO INTEGRATO (IDE): ECLIPSE 15
BIBLIOGRAFIA 17

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Un po' di storia….

Java è un linguaggio di programmazione nato agli inizi degli anni novanta da un gruppo di lavoro della Sun Microsystems. Il gruppo di lavoro guidato da James Gosling si era messa in attività sul nuovo linguaggio in gran segreto a partire dal 1991.

Erano decenni caldi per l'informatica con le comunità scientifiche e le realtà industriali in pieno fermento.

Tra gli anni '70 e la fine degli anni '80 l'hardware aveva subito una delle sue più grandi rivoluzioni: prezzi caduti a picco e performance aumentate straordinariamente.

In quegli anni il C (Dennis Ritchie,1972) era nel pieno della sua diffusione ma la sua struttura procedurale ed il suo approccio "low level" (puntatori, allocazione statica della memoria, etc.) lo rendevano un linguaggio potente ma al tempo stesso impegnativo e di difficile adozione per progetti di sviluppo sempre più complessi e con team di sviluppatori sempre più grandi.

A cavallo fra gli anni '70 e '80 anche un nuovo player era entrato in campo: il linguaggio C++ introdotto da Bjarne Stroustrup. Il C++ si ispirava ai principi della Programmazione Orientata agli Oggetti e durante gli anni '80 si affermò come linguaggio di riferimento per i progetti di sviluppo più grandi e ambiziosi con piena diffusione negli anni '90.

In questo scenario, Sun Microsystem aveva individuato nella realizzazione di un nuovo linguaggio di programmazione uno dei componenti essenziali per catturare la "next-wave" nell'evoluzione dell'informatica. L'idea iniziale per questo nuovo linguaggio è che dovesse servire per scrivere programmi per il controllo di elettrodomestici (TV, frigorifero,...). Il C++ era pur sempre una evoluzione del C e quindi non presentava quelle caratteristiche di semplicità e portabilità che si ritenevano necessarie.

Il piano del gruppo di lavoro alla Sun era dunque quello di realizzare un linguaggio che, come il C++, potesse trarre vantaggio dal nuovo paradigma di programmazione ad oggetti ma che non fosse low-level (il C++ era pur sempre una evoluzione del C), che fosse semplice da usare e si occupasse automaticamente della gran parte dei dettagli di gestione della memoria. Inoltre si richiedeva che fosse un linguaggio che poteva essere eseguito e potesse essere facilmente portato su processori e dispositivi diversi e che non richiedesse compilatori o interpreti troppo sofisticati (i produttori degli elettrodomestici non avrebbero investito risorse in quel settore).

L'idea fu di introdurre un unico, semplice linguaggio intermedio (chiamato byte-code) per il quale potessero facilmente essere scritti interpreti ad-hoc. L'interprete del byte-code Java è detto Java Virtual Machine (JVM). JVM permette di eseguire i programmi scritti nel nuovo linguaggio in qualsiasi sistema

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

operativo supportato dalla tecnologia di Sun. Il target del compilatore non era più il sistema operativo dela macchina host, ma la virtual machine installata su di esso.

Dal gruppo di lavoro in Sun nacque nel 1992 il linguaggio Oak (in italiano "quercia"), nome che successivamente fu cambiato in Java (una varietà di caffè indonesiana; il logo adottato è una tazzina per tale bevanda) per problemi di copyright visto che un altro linguaggio di programmazione Oak esisteva già. Ma la chiave di volta per il successo di Java fu l'idea del gruppo di lavoro di catturare con il loro linguaggio la vera "next-wave" degli anni 90: Internet.

Se un primo momento Sun decise di destinare questo nuovo prodotto alla creazione di applicazioni complesse per piccoli dispositivi elettronici, dopo breve tempo, nel 1993 con l'esplosione di Internet, ci si rese conto che Java poteva essere usato per distribuire applicazioni su Internet e quindi come linguaggio di programmazione per Internet: il byte-code poteva essere distribuito via Web ed essere eseguito sui computer degli utenti. Risultato quindi essere essenziale l'indipendenza dalla piattaforma hardware e quindi vincente l'idea di perseguirla. Nacquero ed ebbero successo le applet: programmi Java eseguibili dentro al browser Web (la JVM installata come plug-in del browser).

Nel giro di pochi mesi il gruppo di lavoro in Sun riuscì a sviluppare un web browser interamente in Java (Sun's HotJava, 1994) con il quale poterono mostrare l'idea che avrebbe legato il nuovo linguaggio alla evoluzione di Internet: le applets, piccole applicazioni che attraverso un browser potevano essere distribuite ed eseguite come mai prima. Nel 1995 Netscape Corporation inserì nel suo browser Navigator la JVM e quindi il supporto per le applet. Fu l'anno della svolta per Java e di una rivoluzione nel mondo di Internet: grazie agli applet le pagine web diventarono interattive a livello client, ovvero le applicazioni vengono eseguite direttamente sulla macchina dell'utente di internet e non su un server remoto. Per esempio gli utenti poterono utilizzare giochi direttamente sulle pagine web e usufruire di chat dinamiche e interattive.

La prima versione beta di Java fu rilasciata a luglio del 1995, mentre la prima release fu annunciata e resa disponibile il 23 gennaio dell'anno 1996.

Con il tempo altre tecnologie soppiantano Java nell'ambito di Internet (e.g. JavaScript) Java rimane comunque uno tra i principali linguaggi per lo sviluppo di applicazioni desktop e distribuite, in particolare in ambiente aziendale (enterprise).

Nel 2010 Sun Microsystems storica e gloriosa sociatà americana è stata assorbito da Oracle. Oggi quindi Java è a tutti gli effetti un prodotto Oracle.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Proprietà del linguaggio Java

Le proprietà generali di Java sono:

- **Sintassi semplice e simile al C/C++:** è un linguaggio piuttosto semplice, con una sintassi simile a quella del C e del C++. Anche il linguaggio di Microsoft C# nato nel 2002 per contrastare Java ha a sua volta una sintassi molto simile. Non si è voluto infatti a livello di sintassi porre alcun sforzo aggiuntivo di apprendimento.
- **Gratuità:** per scrivere applicazioni commerciali non bisogna pagare licenze a nessuno. Il codice Java infatti si può scrivere anche utilizzando un qualsiasi editor di testo come il blocco note e non per forza un complicato IDE con una costosa licenza. Esistono anche tanti strumenti di sviluppi a pagamento che possono accelerare lo sviluppo di applicazioni Java. Tuttavia esistono anche eccellenti prodotti gratuiti ed open source come Eclipse e Netbeans.
- **Robustezza:** grazie ad (i) una gestione delle eccezioni chiara e funzionale, (ii) un meccanismo automatico della gestione della memoria (Garbage Collection) che esonera il programmatore dall'obbligo di dover deallocare memoria quando ce n'è bisogno (punto fra i più delicati nella programmazione), (iii) un compilatore molto «severo» che segnala ogni situazione di utilizzo poco chiara o che esce dall'uso non già previsto del linguaggio, garantendo al programma maggiori chance di corretto funzionamento. Ad esempio il compilatore applica rigide regole sintattiche (il ; alla fine di ogni istruzione) e sui tipi. Jave è un linguaggio fortemente tipato: il programmatore è tenuto a specificare il tipo di ogni variabile, e il compilatore richiede e garantisce che i valori di tali variabili verranno sempre usati in modo coerente rispetto al tipo.
- **Ricca disponibilità di librerie e standardizzazione:** Java possiede un'enorme libreria di classi standard permettendo agli sviluppatori, anche neofiti, di creare applicazioni in breve tempo anche e soprattutto indipendentemente dalla piattaforma in cui si sviluppa. Per esempio, è relativamente semplice gestire finestre di sistema come interfacce grafiche, collegamenti a database, connessioni di rete. Inoltre, grazie alle specifiche di Oracle, non esistono per lo sviluppatore problemi di standardizzazione come compilatori che compilano in modo differente.
- **Facilità di sviluppo:** questo non significa che sia un linguaggio semplice. Sicuramente ad esempio il fatto che siano stata eliminata l'aritmetica dei puntatori è stato senza dubbio una semplificazione ma Java resta un linguaggio molto complesso considerando la sua potenza e tenendo presente che obbliga ad una programmazione ad oggetti pura e rigorosa. Nel tempo il linguaggio arricchito sempre più di

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

nuovi costrutti (ad esempio i Generics o le Annotazioni) che lo hanno reso più difficile da imparare ma al tempo stesso più potente per scrivere programmi di più ampia portata in un tempo relativamente breve. Quindi più propriamente dovremmo parlare di potenza di sviluppo. Java non è facile da imparare ma una volta comprese le logiche fondamentali si ha fra le mani uno strumento potentissimo di sviluppo.

Sulla base delle proprietà generali sopraelencate, i pilastri e le proprietà caratterizzanti del linguaggio sono:

- **Linguaggio fortemente orientato agli oggetti:** Java è più chiaro e schematico di altri linguaggi orientati agli oggetti al punto che praticamente obbliga lo sviluppatore a programmare ad oggetti in modo rigoroso. I paradigmi fondamentali della programmazione ad oggetti (ereditarietà, incapsulamento, polimorfismo) sono più facilmente apprezzabili e comprensibili. Tuttavia il linguaggio è pensato perché essi siano usati in modo stretto e rigoroso. In sostanza lo sviluppatore è costretto a pensare ad oggetti mentre scrive il programma. Tuttavia con il passare del tempo il linguaggio si è evoluto e si è aperto anche ad altri paradigmi di programmazione (es. funzionale).
- **Indipendenza dall'architettura:** grazie al concetto di macchina virtuale, ogni applicazione, una volta compilata, potrà essere eseguita su di una qualsiasi piattaforma (per esempio un PC con sistema operativo Windows o una workstation Unix). Questa è sicuramente la caratteristica più importante di Java. Infatti, nel caso in cui si debba implementare un programma destinato a diverse piattaforme per esempio non ci sarà la necessità di doverlo convertire radicalmente da piattaforma a piattaforma.
- **Byte-code e Java Virtual Machine:** Ciò che rende di fatto possibile l'indipendenza dalla piattaforma è la Java Virtual Machine (JVM) un software che svolge il ruolo di interprete (ma non solo) per i programmi scritti in Java. Si parla di macchina virtuale perché questo software è stato implementato per simulare l'hardware di un calcolatore ipotetico che ha caratteristiche simili (semplificate) a quelle delle architetture hardware più comuni. Il linguaggio Java si basa quindi su un approccio che combina compilazione (in byte-code) e interpretazione da parte di un JVM (del byte-code). Ogni piattaforma ovvero ogni sistema operativo girerà una propria JVM che consentirà l'interpretazione e quindi l'esecuzione del byte-code su quella piattaforma.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Java Programming Model

Il byte code è un linguaggio di basso livello (come l'assembly) che non è legato ad una particolare architettura hardware. Il bytecode in sé può essere però portato e girare attraverso più tipi di piattaforme. L'interprete del byte-code Java `e detto Java Virtual Machine (JVM).

Vediamo questo concetto più in dettaglio con il Java Programming Model (vedi Fig. 1).

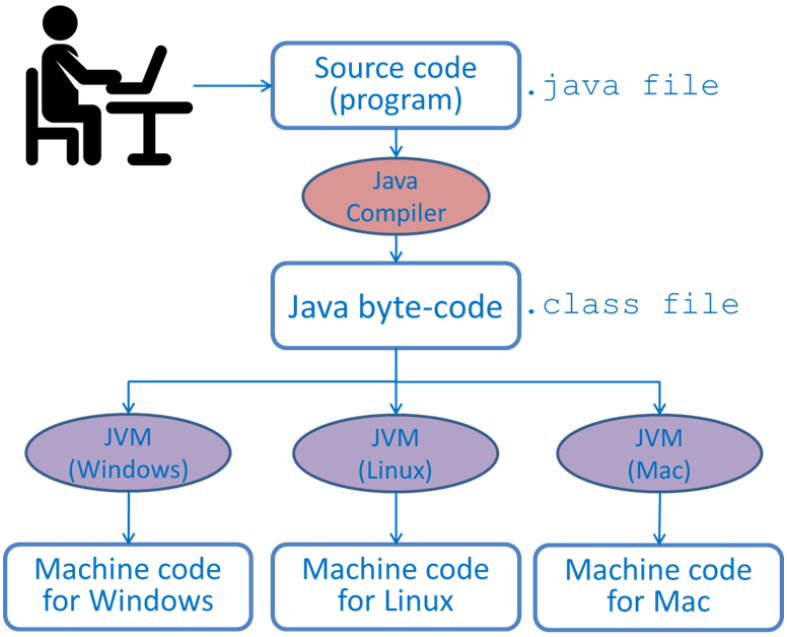


Fig. 1: Java Programming Model

Dopo aver scritto il programma Java, lo sviluppatore deve per prima cosa compilarlo ottenendo così non direttamente un file eseguibile (ovvero la traduzione in un linguaggio macchina del file sorgente scritto in Java) ma un file che contiene la traduzione del listato in un linguaggio detto «byte code». Una volta ottenuto il bytecode, la JVM interpreterà il byte-code ed il programma andrà quindi in esecuzione. Quindi se una piattaforma qualsiasi possiede una JVM ciò sarà sufficiente a renderla esecutrice del bytecode.

Si fa notare che:

- Il compilatore Java è lo stesso per ogni sistema operativo/piattaforma. Mentre l'interprete ovvero la Java Virtual Machine cambia a seconda della piattaforma.
- Il file sorgente deve essere salvato con suffisso .java mentre il file risultato della compilazione avrà lo stesso nome del file sorgente ma con suffisso .class e conterrà il byte-code.

Si potrebbe affermare che il linguaggio macchina sta ad un computer come il bytecode sta ad una JVM. Oltre a permettere l'indipendenza dalla piattaforma la JVM garantisce a java anche (i) la capacità di

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

supportare fra gli altri (i) il multi-threading ovvero capacità di mandare in esecuzione più processi in maniera parallela, (ii) meccanismi di sucurezza molto potenti, (iii) la supervisione del codice da parte del Garrbage Collector.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Il primo programma in Java

Il seguente programma (Fig. 2) visualizza sullo schermo un semplice saluto (Hello World!).

Attenzione: Java è un linguaggio case sensitive, ovvero distingue le lettere maiuscole da quelle minuscole. Quindi scrivere «Class» non è come scrivere «class» e in questo esempio causerà anche un errore in fase di compilazione.

```
1       public class HelloWorld
2       {
3          public static void main (String[] args)
4          {
5              //visualizza un messaggio di saluto
6              System.out.println («Hello World!»);
7          }
8       }
```

Fig. 2: Programma "HelloWorld"

Si tratta del tipico primo programma che si scrive in un nuovo linguaggio di programmazione. Lo usiamo per iniziare a familiarizzare con la sintassi e qualche concetto fondamentale come quello di classe e di metodo. Vedremo anche come compilare e come andare in esecuzione questo programma. Analizziamolo nel dettaglio.

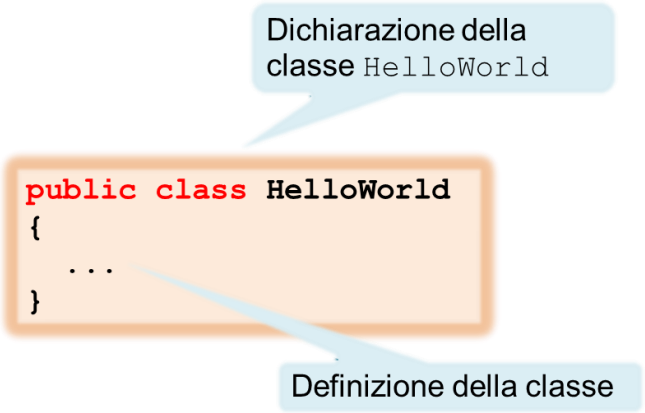


Fig. 3: Dichiarazione della classe HelloWorld

La prima riga con le relative parentesi graffe (righe 1,2,8) dice che stiamo definendo la classe HelloWorld. Come mostrato in Fig. 3, la specifica della classe, ovvero la sua definizione, si trova tra le parentesi graffe. HelloWorld è l'identificatore (ovvero il nome) della classe.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Un programma Java è costituito da un insieme di classi (almeno una); public significa che questa classe è pubblica ovvero può essere utilizzata da qualunque altra classe del programma.

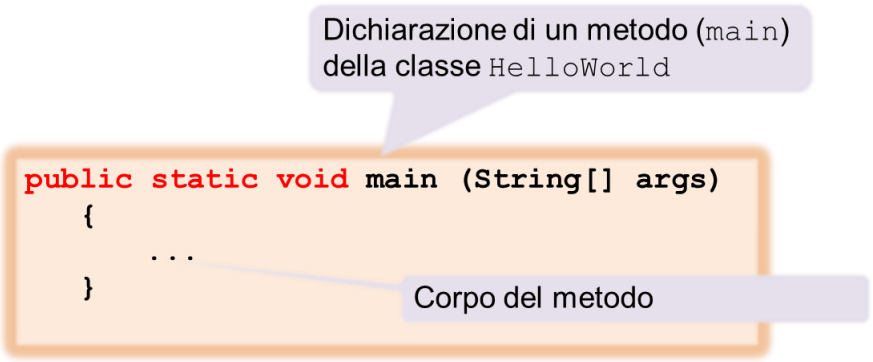


Fig. 4: Dichiarazione del metodo main()

La parte interna della dichiarazione di una classe è dedicata alla definizione dei metodi della classe. Un metodo inteso come l'implementazione di un'operazione è una funzionalità della classe messa disposizione del resto del programma o di altre parti (altri metodi) della stessa classe.

In questo caso alla riga 3 e le relative parentesi graffe (righe 4 e 7) si definisce un metodo della class HelloWorld denominato main (principale) come mostrato in Fig. 4. È un metodo particolare e va tenuto bene a mente. Infatti in ogni applicazione Java questo metodo deve essere definito in ogni applicazione Java perché rappresenta il punto di partenza dell'esecuzione di ogni programma. La prima istruzione che verrà quindi eseguita in fase di esecuzione sarà quella che la JVM troverà subito dopo l'apertura del blocco di codice che definisce questo metodo.

Oltre alla parola «main» la riga contiene le parole «public», «static» e «void». Si tratta di una lista di modificatori, ovvero di parole chiave che caratterizzano la classe come in italiano un aggettivo può caratterizzare un sostantivo.

public significa che questo metodo è visibile e quindi può essere invocato anche al di fuori della classe in cui è definito. Per questo primo programma che è costituito da un unico file ed un'unica classe, questo modificatore non è di alcuna utilità ma più avanti ne risulterà più chiara l'importanza e il significato.

Riguardo a static, la definizione è abbastanza complessa. Per il momento basta sapere che è essenziale per la definizione del metodo main().

void è il tipo di ritorno del metodo. Significa «vuoto» e quindi questo metodo non restituisce nessun tipo di valore. Il metodo main() non deve mail avere un tipo di ritorno diverso da void.

Alla destra di main, in generale alla destra del nome del metodo (detto anche idenificatore del metodo), si definisce sempre una coppia di parentesi tonde che racchiude opzionalmente una lista di parametri (detti anche argomenti del metodo). Il metodo main() in ogni caso vuole sempre come

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

parametro un array di stringhe (gli array li vedremo più avanti). Si noti che args è l'identificatore (nome) dell'array, ed è l'unica parola che può variare nella definizione del metodo main(), anche se per convenzione si usa sempre args.

Il corpo del metodo è racchiuso tra le parentesi graffe.



Fig. 5: Commento in Java

Alla riga 5 è specificato un commento (vedi anche Fig. 5). I commenti servono per rendere più comprensibile il codice. In Java è un commento tutto ciò che si trova a destra di // (una sola riga) o tutto ciò che si trova tra /* e */ (anche su più righe).



Fig. 6: Comando di stampa a video del messaggio "Hello World!"

Alla riga 6 è mostrato il comando (vedi anche Fig. 6) che stamperà a video la stringa Hello World! Anche in questo caso si usano degli argomenti che verranno trattati più avanti. Per il momento basti sapere che si sta usando una libreria standard di Java e invocando un suo metodo chiamato println() sull'oggetto System.out.

System.out è un oggetto che rappresenta il canale di output standard del sistema (la console...). Si tratta di un oggetto istanza di una classe definita altrove e disponibile come libreria standard di Java.

println è un metodo dell'oggetto System.out che stampa un messaggio e va a capo. Come tutti i comandi, println deve essere terminato con punto e virgola dopo la specifica dei parametri previsti, in questo caso la stringa "Hello World!".

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Raccomandazioni per chi viene dalla programmazione procedurale

Per chi viene dalla programmazione procedurale troverà naturale usare Java scrivendo programmi costituiti da un unico listato che sarà posto all'interno di uno stesso metodo che è necessariamente il metodo main che è il metodo invocato per primo e da cui parte il flusso di esecuzione. Quando il programma da realizzare è articolato diventa conveniente identificare sottoproblemi che possono essere risolti individualmente, scrivere sottoprogrammi che risolvono tali sottoproblemi e richiamare i sottoprogrammi dal programma principale (main). In Java i sottoprogrammi si realizzano tramite metodi ausiliari che realizzano quello che in altri linguaggi di programmazione sono chiamati funzioni, procedure oppure (sub)routines. Quindi in aggiunta al main possiamo trovare definiti altri metodi per realizzare tali sottoprogrammi. Essi saranno definiti come «private» specificando così che si trattano di metodi ausiliari ovvero solo invocabile da un altro metodo della stessa classe.

```
public class NomeProgramma
{
   public static void main (String[] args)
   {
     ...
   }

...
   private static <identificatore metodo2> (<lista parametri>)
   {
     ...
   }
...

}
```

Fig. 7: Struttura programma procedurale scritto in Java

Ricapitolando, come mostrato in Fig.7 si ha:

- una sola classe (con nome arbitrario del programma)
- il solo metodo main (scritto esattamente come nell'esempio)
- Il corpo del main conterrà tutti i comandi del programma
- Il main potrà invocare metodi ausiliari definiti come private

Siccome non ci possono essere metodi definiti al di fuori delle classe tutti questi metodo devono esserlo rispetto ad una classe che sarà chiamata con il nome del programma che stiamo scrivendo.

Tuttavia questo uso di Java purché tecnicamente possibile non è quello per cui è stato concepito il linguaggio anzi lo snatura completamente avendo usato la definizione di una classe per scrivere un

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

programma procedurale e definendo le sottoroutine come metodi. Funziona ma è un uso che definirei degenere di questo linguaggio.

Non ha senso imparare il linguaggio Java senza sfruttare il supporto che esso offre alla programmazione ad oggetti.

Quindi vedremo direttamente come creare applicazioni che saranno costituite da un certo numero di classi che daranno vita ad un numero di istanze di oggetti che interagendo realizzeranno quello per cui il programma è stato scritto.

Gli studenti che hanno esperienza con la programmazione procedurale troveranno profonde differenze con i concetti dell'object orientation. Il consiglio per questi ultimi è di non ancorarsi troppo a quello che già si conosce. E' inutile forzare una classe ad avere il ruolo che poteva avere una sottoroutine nella programmazione procedurale. Meglio far finta di partire da zero! Per chi invece già conosce altri linguaggi orientati agli oggetti, le nozioni che seguono dovrebbero risultare chiare e semplici.

Quello che resterà vero è che in un programma Java scritto secondo i dettami dell'object-oriented è il main posto all'interno di costrutto class che sarà nominato con il nome di tutto il programma. Per i primi programmi che presenteremo, identificheremo il programma con una classe avente lo stesso nome e contenente un metodo di nome main. La funzionalità del programma sarà quella realizzata dal metodo main.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

6. Scrivere, compilare ed eseguire un programma in Java

Per scrivere un programma Java si può usare un qualunque editor di testi. Se si lavora con Windows si può usare il semplice "blocco note" oppure Notepad++ (disponibile a http://notepad-plus-plus.org/). Si sconsiglia Wordpad o Word o altri editor che gestiscono stili, formattazioni, etc etc

E' sufficiente aprire l'editor e digitare il programma. La classe dovrà essere contenuta in un file che verrà salvato con lo stesso nome della classe, seguito dal suffisso .java; quindi il programma HelloWorld deve essere scritto in un file chiamato HelloWorld.java.

Per compilare ed eseguire un programma Java avremo bisogno di un compilatore Java e di una Java Virtual Machine (JVM). Entrambi questi strumenti sono forniti dal Java Development Kit (JDK) che si può scaricare dal sito di Oracle [2].

Un modo per eseguire il compilatore Java e la JVM è tramite la console di sistema su Windows ovvero il Prompt dei comandi (o Prompt di DOS). Ci si sposta nella cartella (directory) che contiene il programma e si esegue il compilatore Java tramite il comando javac: javac <nomeFile>.java. Quindi, nel caso del programma HelloWorld dovremo digitare javac HelloWorld.java

A questo punto potremo mandare in esecuzione il programma invocando l'interpretazione della JVM. Basta scrivere dal prompt dei comandi il comando: java <nomeFile>. Il comando java lancia la Java Virtual Machine, cioè l'interprete che è in grado di eseguire i programmi scritti in bytecode.

L'applicazione a meno di errori di codice verrà eseguita dalla JVM e l'esecuzione del programma HelloWorld produrrà l'output a video del messaggio «Hello World!».

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

7. Ambiente di sviluppo integrato (IDE): Eclipse

Nel descrivere come si può compilare ed eseguire un programma Java, abbiamo assunto di avere a disposizione sul nostro calcolatore un ambiente di sviluppo minimale, cioè un editor di testi (per la scrittura dei programmi), un compilatore e un interprete per Java, e naturalmente un interprete di comandi fornito dal sistema operativo che ci consenta di lanciare l'esecuzione del compilatore e dell'interprete con i comandi java e javac, come descritto sopra.

In alternativa, per lo sviluppo di programmi Java si può usare un ambiente integrato di sviluppo, chiamato più brevemente IDE, acronimo di Integrated Development Environment. Un IDE normalmente fornisce un editor guidato dalla sintassi in grado di individuare alcuni errori sintattici già durante la scrittura, e consente di compilare ed eseguire un programma selezionando opportuni pulsanti (o comandi di menu, o combinazioni di tasti) dell'interfaccia grafica. Oltre a integrare editor, compilatore e ambiente di esecuzione, molti IDE forniscono agevolazioni quali il completamento automatico del codice, l'indentazione automatica, l'accesso alla documentazione del linguaggio e delle librerie, la navigazione ipertestuale dei problemi riscontrati, e l'integrazione di potenti strumenti per il collaudo e il debugging. Queste caratteristiche facilitano notevolmente la scrittura di programmi Java e grazie a numerose altre funzionalità disponibili nell'ambiente permettono di gestire in modo efficace lo sviluppo di progetti Java di grandi dimensioni.

Eclipse è fra gli IDE più diffusi che si sta affermando come uno standard di fatto sia in ambito accademico che industriale ed è quello mostrato in Fig. 8.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

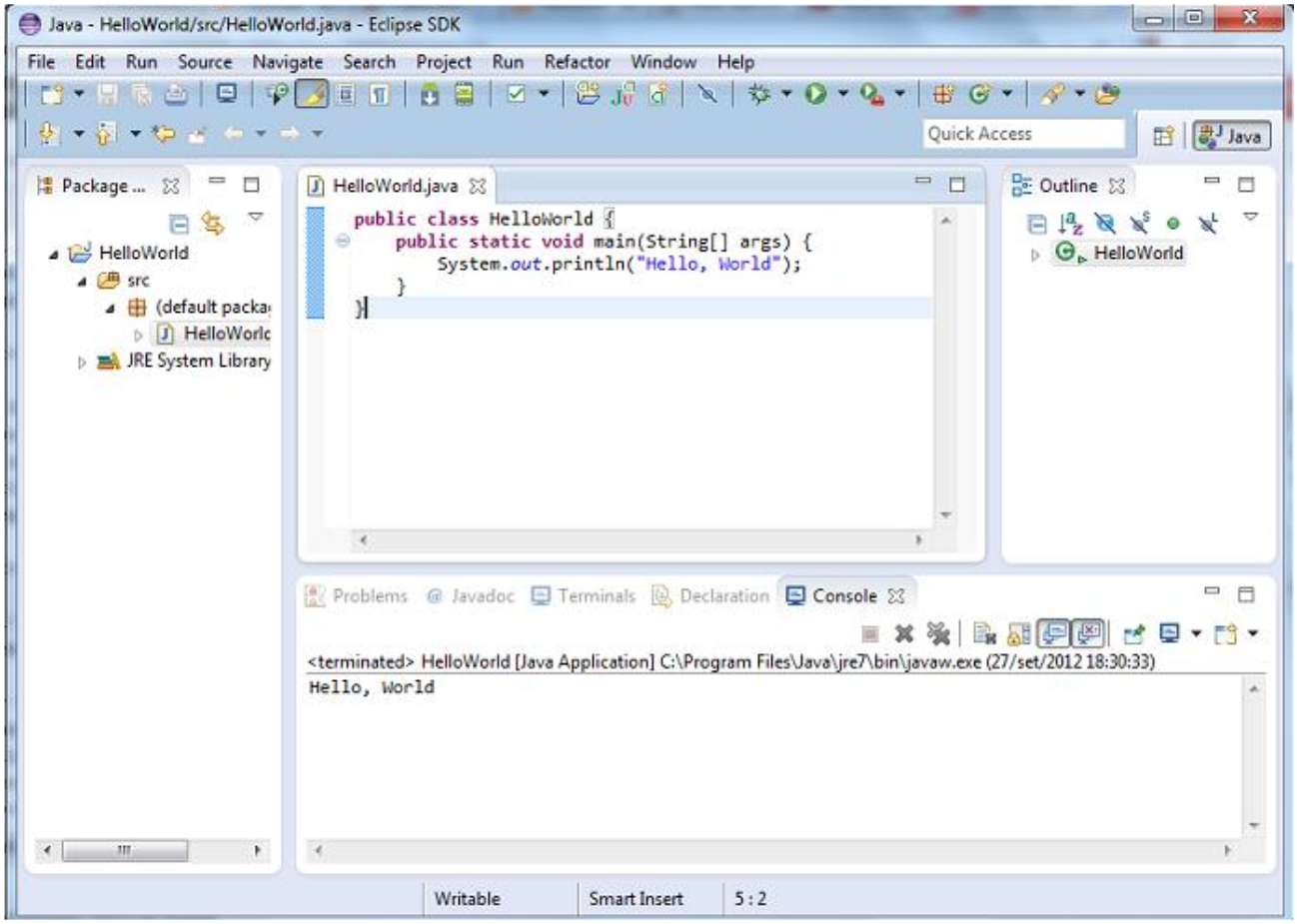


Fig. 8: Ambiente di sviluppo Eclipse

Qui si mostra in particolare quello che le varie finestra mostrano al termine della scrittura e dell'esecuzione del programma HelloWorld: oltre al programma nella finestra dell'editor, al centro, si può intravedere in basso l'output risultante dall'interazione con il programma. In particolare la sotto-finestra (denominata vista) centrale è la finestra dell'editor e consente di scrivere il programma; la vista "Package Explorer" (a sinistra) mostra tutti i file creati; la vista "Outline" (a destra) mostrà alcune informazioni sulla classe corrente; la vista "Problems" (in basso) riporta eventuali errori di compilazione; la vista "Console" consente di interagire con il programma in esecuzione (in questo caso si vede l'output a video prodotto dal programma HelloWorld). Per scaricare Eclipse visitare il sito[3].

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- [1] Il nuovo Java,, Claudio De Sio Cesari, Hoepli Informatica
- [2] https://www.oracle.com/java/technologies/downloads/
- [3] Eclipse http://www.eclipse.org/downloads/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).